SQL Subquery Review and Logic Analysis

Query Review 1

Original Query

```
SELECT first_name, salary
FROM employees
WHERE salary > AVG(salary);
```

Analysis

Result: Oracle returns:

```
ORA-00934: group function is not allowed here
```

Why? `AVG(salary)` is an aggregate function and cannot be used directly in the `WHERE` clause because the `WHERE` clause is evaluated before aggregate calculations are performed.

Correct Version

```
SELECT first_name, salary
FROM employees
WHERE salary >
(
    SELECT AVG(salary)
    FROM employees
);
```

Subquery Type

- Single-row subquery
- Returns one value (average salary)

Execution Order

1. The subquery executes:

   ```
   SELECT AVG(salary) FROM employees;
   ```

2. Oracle calculates the company average salary.
3. The outer query compares each employee salary with the calculated average.
4. Employees earning more than the average salary are returned.

Business Logic

The query finds employees whose salary is greater than the company-wide average salary.

Query Review 2

Original Query

```
SELECT department_name
FROM departments
WHERE department_id =
(
    SELECT department_id
    FROM employees
);
```

Analysis

Result: Oracle returns:

```
ORA-01427: single-row subquery returns more than one row
```

Why? The subquery returns multiple department IDs from the Employees table, while the = operator expects only one value.

Correct Version

```
SELECT department_name
FROM departments
WHERE department_id IN
(
    SELECT department_id
    FROM employees
);
```

Subquery Type

- Multi-row subquery

Execution Order

1. The subquery executes:

   ```
   SELECT department_id FROM employees;
   ```

2. Oracle retrieves all department IDs referenced by employees.
3. The outer query checks each department against the returned list.
4. Departments with at least one employee are returned.

Business Logic

The query returns departments that currently have employees assigned to them.

Query Review 3

Original Query

```
SELECT *
FROM employees
WHERE department_id IN
(
    SELECT salary
    FROM employees
);
```

Analysis

Result: The query may execute but typically returns no rows.

Why? The query compares `department_id` values with `salary` values. These columns represent different business concepts and should not be compared.

Correct Version 1

```
SELECT *
FROM employees
WHERE department_id IN
(
    SELECT department_id
    FROM employees
);
```

Correct Version 2

```
SELECT *
FROM employees
WHERE department_id IN
(
    SELECT department_id
    FROM departments
    WHERE manager_id IS NOT NULL
);
```

Subquery Type

- Multi-row subquery

Execution Order

1. The subquery executes first.
2. Oracle retrieves the values returned by the subquery.
3. The outer query compares employee department IDs against those values.
4. Matching employees are returned.

Business Logic

The corrected query finds employees working in departments that have managers assigned.

Query Review 4

Original Query

```
SELECT first_name
FROM employees
WHERE salary >
(
    SELECT salary
    FROM employees
    WHERE department_id = 90
);
```

Analysis

Result: Oracle returns:

```
ORA-01427: single-row subquery returns more than one row
```

Why? Department 90 may contain multiple employees, causing the subquery to return multiple salary values. The > operator expects only one value.

Correct Version 1 (MAX)

```
SELECT first_name
FROM employees
WHERE salary >
(
    SELECT MAX(salary)
    FROM employees
    WHERE department_id = 10
);
```

Subquery Type

- Single-row subquery

Execution Order

1. The subquery calculates the highest salary in department 10.
2. Oracle returns one salary value.
3. The outer query compares each employee salary against that value.
4. Employees earning more than the highest salary in department 10 are returned.

Business Logic

Find employees whose salary is greater than the highest salary in department 10.

Correct Version 2 (ANY)

```sql
SELECT first_name
FROM employees
WHERE salary > ANY
(
    SELECT salary
    FROM employees
    WHERE department_id = 90
);
```

Subquery Type

- Multi-row subquery

Execution Order

1. The subquery retrieves all salaries from department 90.
2. The outer query compares employee salaries against those values.
3. Employees earning more than at least one employee in department 90 are returned.

Business Logic

Find employees whose salary is greater than at least one employee in department 90.

Correct Version 3 (ALL)

```sql
SELECT first_name
FROM employees
WHERE salary > ALL
(
    SELECT salary
    FROM employees
    WHERE department_id = 30
);
```

Subquery Type

- Multi-row subquery

Execution Order

1. The subquery retrieves all salaries from department 30.
2. The outer query compares employee salaries against all returned values.
3. Employees earning more than every employee in department 30 are returned.

Business Logic

Find employees whose salary is greater than all employees in department 30.

Query Logic Analysis Summary

Query	Subquery Type	Correct	Main Issue
Query 1	Single-row	No	Aggregate function used directly in WHERE
Query 2	Multi-row	No	Single-row operator used with multi-row result
Query 3	Multi-row	No	Comparison between unrelated columns
Query 4	Multi-row	No	Single-row operator used with multi-row result

Final Reflection

1. Why are subqueries useful in enterprise HR reporting systems?

Subqueries allow complex business questions to be answered by first calculating intermediate results and then using those results within a larger query. They are commonly used to identify employees above average salary, top earners, department-level statistics, and other analytical reporting requirements.

2. Difference Between JOIN-Based Thinking and Subquery-Based Thinking

JOIN-based thinking focuses on combining data from multiple related tables.

Example:

```
SELECT e.first_name, d.department_name
FROM employees e
JOIN departments d
ON e.department_id = d.department_id;
```

Subquery-based thinking focuses on calculating a value or result set first and then using that result within another query.

Example:

```
SELECT first_name
FROM employees
WHERE salary >
(
    SELECT AVG(salary)
    FROM employees
);
```

3. Why Do Some Subqueries Produce Runtime Errors?

Common reasons include:

- Using single-row operators (=, >, <) with multi-row subqueries.
- Comparing incompatible data types or unrelated columns.
- Using aggregate functions incorrectly within query clauses.

4. Why Is SQL Execution Order Important?

Oracle logically processes queries in the following order:

```
FROM
WHERE
GROUP BY
HAVING
SELECT
ORDER BY
```

Understanding this order helps explain why certain queries fail and helps developers write more efficient and accurate SQL statements.

5. Why Is Understanding Query Logic More Valuable Than Memorizing Syntax?

Syntax can always be referenced from documentation. However, understanding query logic allows developers to choose the correct subquery type, operator, and filtering strategy while ensuring that business requirements are accurately implemented. Production-level SQL development requires logical reasoning rather than memorization alone.